Module 6 Week A — Core Skills Drill: Text Preprocessing & NER Basics

This drill is due the evening of the concept session (Sunday). It moves you from "I read about text preprocessing and spaCy" to "I can tokenize, clean, and extract entities from text using code." You will practice text preprocessing, part-of-speech (POS) tagging and dependency parsing, Named Entity Recognition (NER), and sentence segmentation. Complete all four tasks, open a PR, and submit your PR URL.

Due: Tonight, after the concept session. Resubmissions are accepted through Saturday of the assignment week.

Setup

1. **Accept the assignment**
2. **Clone your repo:**

   ```
   git clone <your-repo-url>
   cd <your-repo-name>
   ```

3. **Install dependencies:**

   ```
   pip install -r requirements.txt
   ```

   This installs **spaCy** for the first time. If you encounter version conflicts, create a fresh virtual environment first (`python -m venv .venv && source .venv/bin/activate`), then install. (Hugging Face `transformers` is introduced in the Applied Lab, not here.)

4. **Download the spaCy English model:**

   ```
   python -m spacy download en_core_web_sm
   ```

   This downloads a ~12 MB language model. You only need to run this once.

5. **Create a working branch:**

   ```
   git checkout -b drill-6a-text-basics
   ```

All your work goes in `drill.py`. The starter file contains function signatures and docstrings – implement each function as described below.

Task 1: Preprocess a Text Snippet

You are given a raw climate news paragraph and a list of stop words.

Your task:

Implement `preprocess_text(text, stop_words)`:

- Process the text through the spaCy pipeline (`nlp(text)`)
- Iterate over the resulting tokens and keep only those that are not punctuation (`token.is_punct`) and not whitespace (`token.is_space`)
- Lowercase each surviving token (`token.text.lower()`)
- Drop any token whose lowercase form is in the provided `stop_words` set
- Return the cleaned list of lowercase token strings

Use the module-level `nlp` object already loaded in the starter — do not call `spacy.load()` inside the function.

Example:

Input: `"The IPCC released its AR6 report in 2023!"` with stop words `{"the", "its", "in"}`

Expected output: `["ipcc", "released", "ar6", "report", "2023"]`

What the autograder checks: The returned list is lowercase, contains no punctuation-only tokens, and excludes all stop words. The function handles non-ASCII characters (e.g., "Zürich", "café") without crashing.

Task 2: Extract Linguistic Annotations

Your task:

Implement `extract_linguistic_annotations(text)`:

- Use the module-level `nlp` object already loaded in the starter — do not call `spacy.load()` inside the function
- Process the input text through the spaCy pipeline (`nlp(text)`)
- For each token, extract: the token text, its POS tag (`.pos_`), and its dependency label (`.dep_`)
- Return a list of `(token_text, pos_tag, dep_label)` tuples, one per token

Example:

Input: `"Climate scientists published research."`

Expected output (approximate – exact tags depend on spaCy model):

```
[
    ("Climate", "NOUN", "compound"),
    ("scientists", "NOUN", "nsubj"),
    ("published", "VERB", "ROOT"),
    ("research", "NOUN", "dobj"),
    (".", "PUNCT", "punct")
]
```

What the autograder checks: The returned list contains tuples of `(str, str, str)` for each token. POS tags and dependency labels are valid spaCy labels.

Task 3: Run NER and Format Output

Your task:

Implement `extract_entities(text)`:

- Use the module-level `nlp` object already loaded in the starter — do not call `spacy.load()` inside the function
- Process the input text through the spaCy pipeline (`nlp(text)`)
- Extract all named entities from `doc.ents`
- Return a list of `(entity_text, entity_label)` tuples in the order they appear in the text

Example:

Input: `"The United Nations held a summit in Jordan in 2023."`

Expected output:

```
[("The United Nations", "ORG"), ("Jordan", "GPE"), ("2023", "DATE")]
```

What the autograder checks: The returned list contains `(str, str)` tuples. Entity text and labels match spaCy's output for the given input.

Task 4: Split Text Into Sentences

Your task:

Implement `split_into_sentences(text)`:

- Use the module-level `nlp` object already loaded in the starter — do not call `spacy.load()` inside the function
- Process the input text through the spaCy pipeline (`nlp(text)`)
- Use spaCy's `doc.sents` to iterate over detected sentences
- For each sentence, return its text with leading and trailing whitespace stripped
- Return a list of sentence strings, in the order they appear in the text

Example:

Input: `"The IPCC released its latest report. Climate scientists called it urgent."`

Expected output:

```
[
    "The IPCC released its latest report.",
    "Climate scientists called it urgent."
]
```

Sentence segmentation is rule-based + model-driven in spaCy: `doc.sents` looks at punctuation, capitalization, and learned cues to find sentence boundaries. It is not always perfect — abbreviations like "Dr." or bullet lists can confuse it — but for well-formed news text it is reliable.

What the autograder checks: The returned list contains the correct number of sentences, in order, with no leading/trailing whitespace. Single-sentence input returns a one-element list.

Submission

1. Commit and push your completed `drill.py`:

   ```
   git add drill.py
   git commit -m "Complete drill 6A tasks"
   git push origin drill-6a-text-basics
   ```

2. Open a pull request from `drill-6a-text-basics` to `main`.
3. Confirm the autograder passes (green check on your PR).
4. Paste your PR URL into TalentLMS -> Module 6 Week A -> Drill 6A to submit this assignment.